

The Ultimate Front-End Guide

Part 1: Deep-Dive JavaScript, React, and Next.js

Generated for Technical Mastery

May 20, 2026

Contents

1	Advanced JavaScript	2
1.1	Explain Closures in JavaScript with an example.	2
1.2	How does the Event Loop work in JavaScript?	2
1.3	Explain <code>var</code> , <code>let</code> , and <code>const</code> (Hoisting and Scope).	3
1.4	How does the <code>this</code> keyword work?	3
1.5	Explain Call, Apply, and Bind.	4
1.6	What is Event Delegation?	4
1.7	Implement Debouncing and Throttling.	4
1.8	What is Prototypal Inheritance?	4
1.9	Explain WeakMap and WeakSet.	5
1.10	What are Generators?	5
2	React.js Fundamentals & Advanced	6
2.1	What is the Virtual DOM and how does reconciliation work?	6
2.2	Explain the <code>useEffect</code> hook and its dependency array.	6
2.3	Explain <code>useMemo</code> and <code>useCallback</code>	6
2.4	What are Custom Hooks?	7
2.5	Explain Prop Drilling and the Context API.	7
2.6	Controlled vs. Uncontrolled Components.	7
2.7	What is <code>useReducer</code> ?	7
2.8	Explain React Portals.	7
2.9	What is <code>useLayoutEffect</code> ?	7
3	Next.js (App & Pages Router)	8
3.1	Explain SSR, SSG, ISR, and CSR in Next.js.	8
3.2	App Router vs. Pages Router in Next.js 13+	8
3.3	What are Server Actions in Next.js?	8
3.4	How does Middleware work in Next.js?	8
3.5	Explain Next.js Image Optimization (<code>next/image</code>).	9

Chapter 1

Advanced JavaScript

1.1 Explain Closures in JavaScript with an example.

Context: Closures are one of the most frequently asked JS questions, demonstrating a deep understanding of lexical scoping.

Answer: A closure is a feature in JavaScript where an inner function has access to the outer (enclosing) function's variables—a scope chain. It has three scope chains: it has access to its own scope, it has access to the outer function's variables, and it has access to the global variables. Crucially, a closure remembers the environment in which it was created, *even after the outer function has finished executing*.

Why it matters: Closures are used for data privacy (emulating private methods), partial application/currying, and event handlers.

Example (Data Privacy):

```
1 function createCounter() {
2   let count = 0; // 'count' is essentially private
3   return {
4     increment: function() {
5       count++;
6       return count;
7     },
8     decrement: function() {
9       count--;
10      return count;
11    }
12  };
13 }
14
15 const myCounter = createCounter();
16 console.log(myCounter.increment()); // 1
17 console.log(myCounter.increment()); // 2
18 console.log(myCounter.count); // undefined (cannot access directly)
```

1.2 How does the Event Loop work in JavaScript?

Context: Understanding the Event Loop is vital for mastering asynchronous JavaScript.

Answer: JavaScript is single-threaded, meaning it can only execute one piece of code at a time on the main thread (Call Stack). To handle asynchronous operations without blocking the thread, it uses the Event Loop.

1. **Call Stack:** Code is pushed here and executed synchronously.
2. **Web APIs:** Asynchronous tasks (like `setTimeout`, DOM events, `fetch`) are handed off to the browser's Web APIs to execute in the background.

3. Task Queues:

- **Microtask Queue:** Holds Promises (`.then`, `.catch`, `async/await`) and `MutationObserver`. This has the highest priority.
- **Macrotask Queue:** Holds `setTimeout`, `setInterval`, etc.

4. **The Event Loop:** It continuously checks if the Call Stack is empty. If it is empty, it first processes *all* tasks in the Microtask Queue. Once the Microtask Queue is empty, it processes *one* task from the Macrotask Queue, then checks the Microtask Queue again.

Example:

```
1 console.log('Start'); // 1. Call stack
2
3 setTimeout(() => {
4   console.log('Timeout'); // 4. Macrotask Queue
5 }, 0);
6
7 Promise.resolve().then(() => {
8   console.log('Promise'); // 3. Microtask Queue
9 });
10
11 console.log('End'); // 2. Call stack
12
13 // Output: Start, End, Promise, Timeout
```

1.3 Explain var, let, and const (Hoisting and Scope).

Answer:

- **var:** Function-scoped. It is hoisted to the top of its scope and initialized with `undefined`.
- **let:** Block-scoped. It is hoisted to the top of its block but is *not* initialized. Accessing it before declaration results in a `ReferenceError` (Temporal Dead Zone). It can be reassigned.
- **const:** Block-scoped. Same hoisting behavior as `let` (Temporal Dead Zone). It *cannot* be reassigned. However, if a `const` holds an object or array, the properties/elements inside can still be mutated.

1.4 How does the this keyword work?

Answer: The value of `this` depends entirely on *how* a function is invoked, not where it is defined.

1. **Implicit Binding:** `obj.method()` → `this` points to `obj`.
2. **Explicit Binding:** `func.call(obj)`, `func.apply(obj)`, `func.bind(obj)` → `this` points to `obj`.
3. **New Binding:** `new ConstructorFunc()` → `this` points to the newly created empty object.
4. **Default Binding:** A standalone function call `func()` → `this` points to the global object (`window` in browsers) or `undefined` in strict mode.
5. **Arrow Functions:** They do not have their own `this`. They inherit `this` lexically from the surrounding scope at the time they are defined.

1.5 Explain Call, Apply, and Bind.

Answer: All three are methods used to explicitly set the `this` context of a function.

- `call()`: Invokes the function immediately and passes arguments one by one (comma-separated).
- `apply()`: Invokes the function immediately but passes arguments as an array.
- `bind()`: Does *not* invoke the function immediately. It returns a *new* function with `this` permanently bound to the provided object, to be called later.

1.6 What is Event Delegation?

Answer: Event delegation is a technique involving adding a single event listener to a parent element rather than adding individual listeners to multiple child elements. It leverages the concept of **Event Bubbling** (events propagating from the target element up the DOM tree).

Benefits: Saves memory, improves performance, and automatically handles dynamically added child elements.

```
1 // Instead of attaching to 100 <li> elements:
2 document.getElementById('parent-ul').addEventListener('click', function(e) {
3     if (e.target && e.target.nodeName === 'LI') {
4         console.log('List item clicked: ', e.target.id);
5     }
6 });
```

1.7 Implement Debouncing and Throttling.

Answer: Both are techniques to control how many times a function gets executed.

- **Debounce:** Delays the execution of a function until a certain amount of time has passed *since the last time it was triggered*. Perfect for search input fields (don't search until the user stops typing).
- **Throttle:** Ensures a function is called *at most once* within a specified time interval, regardless of how many times the event fires. Perfect for scroll or resize events.

```
1 function debounce(func, delay) {
2     let timeoutId;
3     return function(...args) {
4         clearTimeout(timeoutId);
5         timeoutId = setTimeout(() => {
6             func.apply(this, args);
7         }, delay);
8     };
9 }
```

1.8 What is Prototypal Inheritance?

Answer: JavaScript is an object-based language based on prototypes, rather than being class-based. Every JavaScript object has an internal hidden property called `[[Prototype]]` (accessed via `__proto__`) that points to another object. When you try to access a property on an object,

if it doesn't exist, JS looks at the object's prototype. If it's not there, it looks at the prototype's prototype, forming a **prototype chain**, until it finds the property or reaches `null`.

1.9 Explain WeakMap and WeakSet.

Answer: Similar to `Map` and `Set`, but with key differences: 1. They only accept **Objects** as keys (or values in `WeakSet`), not primitive values. 2. They hold "weak" references to the objects. If there are no other references to the object used as a key, it will be automatically Garbage Collected. This prevents memory leaks, making them ideal for caching data related to DOM nodes that might be removed from the page.

1.10 What are Generators?

Answer: Generators are special functions that can be paused and resumed, yielding multiple values over time instead of returning a single value. Defined with `function*` and use the `yield` keyword.

```
1 function* idGenerator() {
2   let id = 1;
3   while (true) {
4     yield id++; // Pauses execution here and returns the value
5   }
6 }
7 const gen = idGenerator();
8 console.log(gen.next().value); // 1
9 console.log(gen.next().value); // 2
```

Chapter 2

React.js Fundamentals & Advanced

2.1 What is the Virtual DOM and how does reconciliation work?

Answer: The Virtual DOM is a lightweight JavaScript representation of the actual DOM. Updating the real DOM is slow and expensive. React uses the Virtual DOM to optimize updates.

The Reconciliation Process:

1. When state or props change, React creates a new Virtual DOM tree.
2. It compares this new tree with the previous Virtual DOM tree. This comparison process is called **diffing**.
3. React calculates the minimum number of changes required to update the real DOM.
4. It batches these changes and applies them to the real DOM in a single update (**patching**).

React uses `key` props in lists to optimize the diffing algorithm, allowing it to identify which specific items changed, moved, or were deleted without re-rendering the whole list.

2.2 Explain the `useEffect` hook and its dependency array.

Answer: `useEffect` allows you to perform side effects (data fetching, subscriptions, manual DOM manipulation) in function components.

The **dependency array** controls when the effect runs:

- **No array:** Effect runs after *every* render.
- **Empty array []:** Effect runs exactly *once* after the initial render (like `componentDidMount`).
- **Array with dependencies [var1]:** Effect runs on initial render, and subsequently only if `var1` has changed since the last render.

Returning a function from `useEffect` creates a **cleanup function**, which runs before the component unmounts or before the effect runs again (like `componentWillUnmount`).

2.3 Explain `useMemo` and `useCallback`.

Answer: Both hooks are used for performance optimization to prevent unnecessary re-renders.

- **`useMemo`:** Caches the *result* of an expensive calculation. It only recalculates the value when one of its dependencies changes.
- **`useCallback`:** Caches a *function definition* between renders. This is crucial when passing callback functions as props to memoized child components (using `React.memo`). Without it, a new function reference is created on every parent render, breaking the child's memoization.

2.4 What are Custom Hooks?

Answer: Custom hooks are JavaScript functions whose names start with “use” and that call other React Hooks. They are used to extract reusable stateful logic (not state itself) across components. For example, a `useWindowSize()` hook or a `useFetch(url)` hook.

2.5 Explain Prop Drilling and the Context API.

Answer: **Prop drilling** is passing props through many layers of intermediate components that don't actually need the data, just to get it to a deeply nested child component. The **Context API** solves this by creating a global state for a component subtree. A `Provider` component supplies the data, and any deeply nested child can read it using the `useContext` hook without needing props passed down manually.

2.6 Controlled vs. Uncontrolled Components.

Answer:

- **Controlled:** Form input data is handled completely by React state (`useState`). The input's `value` attribute is tied to state, and updates occur via an `onChange` handler.
- **Uncontrolled:** Form data is handled by the DOM itself. You read the values when needed (e.g., on form submit) using a `ref` (via `useRef`). Good for integrating with non-React code.

2.7 What is `useReducer`?

Answer: An alternative to `useState` for complex state logic involving multiple sub-values or when the next state depends on the previous one. It uses a Redux-like reducer function (`state, action`) => `newState` and a `dispatch` function.

2.8 Explain React Portals.

Answer: Allows you to render children into a DOM node that exists outside the DOM hierarchy of the parent component. This is essential for components like Modals, Tooltips, and Drop-downs. It allows them to visually break out of a parent container that might have `overflow: hidden` or restrictive z-index stacking contexts, while still behaving logically like a React child.

2.9 What is `useLayoutEffect`?

Answer: Identical signature to `useEffect`, but it fires *synchronously* immediately after all DOM mutations, but *before* the browser has a chance to paint the screen. Use this strictly when you need to read layout from the DOM (like measuring an element's width) and synchronously re-render to prevent a visual flicker.

Chapter 3

Next.js (App & Pages Router)

3.1 Explain SSR, SSG, ISR, and CSR in Next.js.

Answer:

- **SSG (Static Site Generation):** HTML is generated at *build time*. It is incredibly fast because the CDN serves pre-rendered HTML. Used for blogs, marketing pages.
- **SSR (Server-Side Rendering):** HTML is generated on the server *per request*. Great for dynamic data that must be SEO friendly (e.g., a user dashboard).
- **ISR (Incremental Static Regeneration):** Allows you to update static pages *after* build time in the background. If a request comes in after a defined time, it serves the stale page, triggers a background rebuild, and subsequent requests get the new page.
- **CSR (Client-Side Rendering):** Standard React behavior where JS fetches data and renders in the browser.

3.2 App Router vs. Pages Router in Next.js 13+

Answer:

- **Pages Router:** Routing is based on the `pages/` directory. Data fetching uses explicit functions (`getServerSideProps`). Components default to client-side rendering (with initial server HTML).
- **App Router:** Routing is based on the `app/` directory using folders. Introduces **React Server Components (RSC)** as the default. Server components run exclusively on the server and ship zero JavaScript to the client. You opt-in to interactivity using the `"use client"` directive.

3.3 What are Server Actions in Next.js?

Answer: Asynchronous functions that execute on the server but can be called directly from Client Components (e.g., in a form `action` attribute). They eliminate the need to write manual, boilerplate API endpoints for basic database mutations.

3.4 How does Middleware work in Next.js?

Answer: Middleware is code that executes *before* a request is completed. You can run code, modify the request/response, rewrite paths, or redirect users based on headers or cookies. It is highly optimized and runs on the Edge, making it perfect for authentication checks and A/B testing.

3.5 Explain Next.js Image Optimization (next/image).

Answer: The `<Image>` component provides automatic performance optimizations: 1. Serves correctly sized images based on the user's device screen size. 2. Automatically converts images to modern, smaller formats like WebP or AVIF. 3. Lazy-loads images by default (only loading them when they enter the viewport) to drastically improve Core Web Vitals (specifically Largest Contentful Paint - LCP).